Project Report

Automatic Color Sorting Robotic Arm

Contents

Abstract....2
References 3
Team Member Details 3
Hardware Component Used 3
Sustainable Development Goals Achieved 3
Code....6

Abstract: The **Automatic Color Sorting Robotic Arm** is a microcontroller-based system designed to identify and sort objects based on their color. Coupled with Raspberry pi for color detection via regular webcam, the system captures the color of a stationary object placed within its field of view. Once the color is determined, the robotic arm, driven by servo motors, is programmed to move the object into its corresponding bin.

The system is designed to operate autonomously, making decisions based on real-time color analysis. It demonstrates fundamental concepts in embedded systems, image-based processing, and automation. Applications for this project include smart recycling systems, industrial automation, and educational demonstrations.

This project balances hardware and software challenges and explores the practical integration of color sensing, real-time control, and mechanical actuation—all powered by affordable components and open-source IDEs (Arduino and Python).

References

Arduino Base Code and Python Base Code:

GitHub - khan-tahir/Vision-Assisted-Robotic-Arm: This project is a fusion of robotics and computer vision, utilizing Arduino Uno R3 and OpenCV to automate object sorting by color. It showcases innovative technology while emphasizing our commitment to sustainability and responsible innovation.

Referred Paper:

https://www.ijraset.com/best-journal/vision-assisted-robotic-arm-using-arduino

Team Member Details

- Arnav Panwar – 23BAI0147
- Bhavye Nijhawan – 23BAI0100
- Aditya Singh – 23BAI0088

Hardware Component Used

- Breadboard
- Arduino Uno (controlling servos)
- Raspberry pi-3 model B
- Jumper wires
- MB102 board power supply
- 4 SG90 Motors
- DIY Robotic Arm
- Webcam
- 9v battery (optional but recommended)
- Power Adapter
- External light source (environment dependent)

Sustainable Development Goals Achieved

SDG 12 – Responsible Consumption and Production

- **Efficient sorting of recyclable materials (e.g., plastics by color) reduces manual labor, increases recycling rates, and minimizes contamination.**
- **Supports waste segregation at source in industrial or recycling plants.**
- **Encourages circular economy practices by automating reusability sorting.**

SDG 9 – Industry, Innovation, and Infrastructure

- **Represents innovation in automation and smart manufacturing.**

- **This contributes to building more resilient, efficient industrial processes.**
- **Promotes the integration of robotics in supply chains and factories.**

SDG 8 – Decent Work and Economic Growth

- **Automates repetitive work and reduces risk of injury of human labor in sorting lines.**
- **Creates demand for technical jobs (robot maintenance, programming, AI integration).**
- **Encourages upskilling in robotics and automation fields.**

Code

Arduino Code:

```
#include <Servo.h>

Servo baseServo;
Servo shoulderServo;
Servo elbowServo;
Servo gripServo;

int basePin = 9;
int shoulderPin = 10;
int elbowPin = 11;
int gripPin = 12;

void moveArmToRed();
void moveArmToGreen();
void moveArmToBlue();
void moveArmToDefault();
void moveArmToPickup();
void moveServoSmooth(Servo &servo, int targetAngle, int delayTime);

void setup(){
  Serial.begin(9600);
  baseServo.attach(basePin);
  shoulderServo.attach(shoulderPin);
  elbowServo.attach(elbowPin);
  gripServo.attach(gripPin);
  moveArmToDefault();
}

void loop(){
  while (Serial.available()){
    char color = Serial.read();
    Serial.println(color);

    if (color == 'R'){
      moveArmToPickup();
      moveArmToRed();
    }
    else if (color == 'G'){
      moveArmToPickup();
      moveArmToGreen();
```

```
    }
    else if (color == 'B'){
      moveArmToPickup();
      moveArmToBlue();
    }
    else if (color == 'D'){
      moveArmToDefault();
    }
  }
}

void moveArmToRed(){
  Serial.println("Moving to Red Object...");
  moveServoSmooth(shoulderServo, 140, 7);
  moveServoSmooth(elbowServo, 170, 7);
  delay(500);
  moveServoSmooth(baseServo, 45, 5);
  delay(500);
  moveServoSmooth(gripServo, 0, 2);
  delay(200);
}

void moveArmToGreen(){
  Serial.println("Moving to Green Object...");
  moveServoSmooth(shoulderServo, 140, 7);
  moveServoSmooth(elbowServo, 170, 7);
  delay(500);
  moveServoSmooth(baseServo, 150, 5);
  delay(500);
  moveServoSmooth(gripServo, 0, 2);
  delay(200);
}

void moveArmToBlue(){
  Serial.println("Moving to Blue Object...");
  moveServoSmooth(shoulderServo, 140, 10);
  moveServoSmooth(elbowServo, 170, 10);
  delay(500);
  moveServoSmooth(baseServo, 90, 5);
  delay(500);
  moveServoSmooth(gripServo, 0, 2);
  delay(200);
}

// Function to move the arm to default position
```

```
void moveArmToDefault(){
  Serial.println("Returning to Default Position...");
  moveServoSmooth(baseServo, 90, 5);
  moveServoSmooth(shoulderServo, 145, 10);
  moveServoSmooth(elbowServo, 120, 10);
  moveServoSmooth(gripServo, 90, 2);
}

void moveArmToPickup(){
  Serial.println("Moving to Pickup...");
  moveServoSmooth(shoulderServo, 140, 10);
  moveServoSmooth(elbowServo, 170, 10);
  moveServoSmooth(baseServo, 0, 5);
  moveServoSmooth(gripServo, 0, 5);
  moveServoSmooth(elbowServo, 130, 7);
  moveServoSmooth(shoulderServo, 180, 7);
  delay(500);
  moveServoSmooth(gripServo, 120, 2);
}

// Function to move servo smoothly
void moveServoSmooth(Servo &servo, int targetAngle, int delayTime){
  int currentAngle = servo.read();
  if (currentAngle < targetAngle){
    for (int i = currentAngle; i <= targetAngle; i++){
      servo.write(i);
      delay(delayTime);
    }
  } else {
    for(int i = currentAngle; i >= targetAngle; i--){
      servo.write(i);
      delay(delayTime);
    }
  }
}
```

Python Code:

```
import serial
import cv2
import numpy as np
import time

# Initialize serial communication with Arduino
arduino = serial.Serial('COM6', 9600, timeout=1)
time.sleep(2)  # Wait for Arduino to initialize
arduino.reset_input_buffer()

cap = cv2.VideoCapture(1)  # Change to 0 for laptop camera
cap.set(cv2.CAP_PROP_AUTO_EXPOSURE, 0.25)
cap.set(cv2.CAP_PROP_EXPOSURE, -5)
cap.set(cv2.CAP_PROP_BRIGHTNESS, 35)
cap.set(cv2.CAP_PROP_CONTRAST, 70)

if not cap.isOpened():
    print("Error: Could not open camera.")
    exit()

# Define the region of interest (ROI)
ROI_X, ROI_Y, ROI_W, ROI_H = 200, 130, 200, 200  # Adjust as needed
last_color = None  # Store last sent color

def detect_and_draw(mask, color_name, color_bgr, frame, roi_offset):
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    if contours:
        largest_contour = max(contours, key=cv2.contourArea)
        if cv2.contourArea(largest_contour) > 500:
            x, y, w, h = cv2.boundingRect(largest_contour)
            x += roi_offset[0]  # Adjust coordinates based on ROI
            y += roi_offset[1]
            cv2.rectangle(frame, (x, y), (x + w, y + h), color_bgr, 2)
            cv2.putText(frame, color_name, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX,
0.6, color_bgr, 2)
            return True
    return False

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error: Could not read frame.")
```

```
        break

    # Draw ROI on the frame
    cv2.rectangle(frame, (ROI_X, ROI_Y), (ROI_X + ROI_W, ROI_Y + ROI_H), (0, 255,
255), 2)

    # Crop to ROI
    roi = frame[ROI_Y:ROI_Y + ROI_H, ROI_X:ROI_X + ROI_W]

    hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
    gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)

    # Define color ranges
    red_lower1, red_upper1 = np.array([0, 100, 100]), np.array([10, 255, 255])
    red_lower2, red_upper2 = np.array([170, 100, 100]), np.array([180, 255, 255])
    green_lower, green_upper = np.array([45, 100, 100]), np.array([75, 255, 255])
    blue_lower, blue_upper = np.array([100, 100, 100]), np.array([130, 255, 255])
    black_lower, black_upper = np.array([0, 0, 0]), np.array([180, 255, 40])

    # Create masks
    mask_red = cv2.bitwise_or(cv2.inRange(hsv, red_lower1, red_upper1),
cv2.inRange(hsv, red_lower2, red_upper2))
    mask_green = cv2.inRange(hsv, green_lower, green_upper)
    mask_blue = cv2.inRange(hsv, blue_lower, blue_upper)
    mask_black = cv2.inRange(hsv, black_lower, black_upper)

    _, black_mask_gray = cv2.threshold(gray, 40, 255, cv2.THRESH_BINARY_INV)
    mask_black = cv2.bitwise_and(mask_black, black_mask_gray)

    # Determine object color within ROI
    color_detected = None
    roi_offset = (ROI_X, ROI_Y)
    if detect_and_draw(mask_red, "Red", (0, 0, 255), frame, roi_offset):
        color_detected = 'R'
    elif detect_and_draw(mask_green, "Green", (0, 255, 0), frame, roi_offset):
        color_detected = 'G'
    elif detect_and_draw(mask_blue, "Blue", (255, 0, 0), frame, roi_offset):
        color_detected = 'B'
    elif detect_and_draw(mask_black, "Black", (50, 50, 50), frame, roi_offset):
        color_detected = 'D'

    print(f"Red: {cv2.countNonZero(mask_red)}, Green:
{cv2.countNonZero(mask_green)}, "
          f"Blue: {cv2.countNonZero(mask_blue)}, Black:
{cv2.countNonZero(mask_black)}")
```

```
    if color_detected and color_detected != last_color:
        arduino.write(color_detected.encode())
        print(f"Sent: {color_detected}")
        last_color = color_detected
        time.sleep(0.5)
        response = arduino.readline().decode().strip()
        if response:
            print(f"Arduino Response: {response}")

    # input key for closing the program (q)
    key = cv2.waitKey(1) & 0xFF
    cv2.imshow('Frame', frame)  # Show full frame with ROI outline
    cv2.imshow('ROI', roi)  # Show cropped ROI

    if key == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
arduino.close()
```